Sep Hero — Zybo Z7 + BoosterPack — Diagrama funcional (qué corre en Vivado y qué en Vitis)

Hardware físico
Zybo Z7 · BoosterPack · tarjeta de extensión

PL · Vivado (VHDL + IP Xilinx) · FCLK_CLK0 = 50 MHz

PS · Zynq-7000 ARM Cortex-A9 (CPU0) · **Vitis**: C bare-metal (standalone)

AXI4-Lite

M_AXI_GP0 (maestro)

Bloques fijos PS | *Drivers (C)* | *Módulos de servicio (C)* | *Núcleo del juego (C)*

Booster S1 · S2 · JOY_SEL
T15 · P14 · R14
OK · atrás/pausa · OK

s1, s2, joy_sel

debouncer_axi_0
IP propio · VHDL · AXI4-Lite
antirrebote + detector de flanco (3 botones)
irq en cada flanco de subida

R: EDGE[2:0], STATE[2:0]
W: CTRL, DEB_MS, BTN_MASK

debouncer_axi.c/.h *(propio)*
debouncer_init(Debouncer *dev, const DebCfg *cfg)
debouncer_get_edges(dev)

edges

input.c · botones
deb_isr() lee EDGE:
S1 = OK · S2 = atrás/pausa · SEL = OK

OK, ATRÁS, PAUSA

irq_deb

Joystick X / Y
←, ↑, ↓, → = *4 carriles*

VRx, VRy

Acelerómetro X / Y / Z

aX, aY, aZ

Micrófono

audio

ADC SPI
tarjeta de extensión
10 bit · multicanal

SPI

axi_quad_spi_1
maestro SPI del ADC
Tx: comando de canal
Rx: muestra 10 bit

XSpi_Transfer:
canal ↔ 2 bytes

ADC.c (curso) · XSpi
read_joyx() · read_joyy()
read_acx/y/z()
read_MIC()

joy x, y

input.c · flechas (cada 2 ms)
eje dominante + histéresis → ← ↑ ↓ →
flanco CENTRO→dir: {lane, song_ms} a la cola

hit{lane, song_ms}

acc[3], mic

sensors.c (cada 10 ms)
shake: |a| − 1 g > UMBRAL_SHAKE
mic: pico a pico → público

shake, crowd_lvl

OPT3001 · sensor de luz
I²C 0x44 · SCL V17 · SDA V18
pin INT (umbral lux)

SDA / SCL

INT

axi_iic_0
maestro I²C

W: CONFIG 0x01, LOW_LIMIT 0x02, HIGH_LIMIT 0x03 · R: RESULT 0x00 (lux)

I2C.c (curso) + **opt3001.c** (propio)
XIic_Send / XIic_Recv
opt3001_init(Opt3001 *s, lo, hi)

lux

theme.c
gpio_isr() → theme_isr():
lee RESULT (lux), cambia
CLARO ⇄ OSCURO
y re-arma LOW / HIGH_LIMIT

theme

axi_gpio_1 (1 canal, entrada)
CH1[0] = OPT_INT
ip2intc_irpt

R: DATA (OPT_INT)
W: IER, ISR

XGpio (xgpio.h)
XGpio_DiscreteRead (OPT_INT)
XGpio_InterruptEnable

OPT_INT

LCD 128×128
SCLK V13 · MOSI J15 · CS W16
RS V12 · RST H15

SPI MOSI / SCLK / CS
RS, RST

axi_quad_spi_0 + axi_gpio_0
SPI del LCD · GPIO RS / RST

W: bytes RGB565 · DC · RST

LCD_Driver.c · LCD_GUI.c (curso)
XSpi + XGpio · fuentes 8–24
GUI_DrawRectangle · GUI_DisString_EN

sprites, texto

render.c
pista ← ↑ ↓ →, receptores y notas-flecha
(sprites 18×18) · HUD · menús · azul en star

Screen, GameState*, theme, star

game_engine.c

```
typedef enum { IZQ, ARRIBA, ABAJO, DER } Lane; // ← ↑ ↓ →
typedef struct { u32 t_ms; Lane lane; u16 freq; u16 dur; } Note;
typedef struct { char name[16]; u16 bpm; u16 n; Note *chart; }
Song;
typedef struct { u32 score; u16 streak; u8 star_bar;
  u32 star_ms; s8 rock; u8 theme; } GameState;
```

- hit = flecha del joystick == carril de la nota
  y |t_hit − t_nota| ≤ HIT_WIN_MS
- puntos = PTS_NOTA × (star ? 2 : 1)
- streak ≥ STAR_STREAK → barra de poder llena
- barra llena + shake → **STAR POWER** 20 s: ×2, notas y bordes azules
- rock meter: acierto +1 / fallo −2; rock = 0 → GAME OVER
- público (mic) en el HUD · grito en pantalla final → bonus encore

SONG_END, ROCK_0, STAR_ON

game_fsm.c · main()

INIT → MENU → SELECCIÓN DE NIVEL → JUEGO ⇄ PAUSA → GAME OVER → MENU
MENU también lleva a RÉCORDS y a CONFIG (UART)
(detalle en la página 3)

```
int main(void) {
  init_all(&ctx);  // GIC, timers, SPI, IIC, SD
  while (1) {
    if (ctx.frame_flag) {
      fsm_step(&ctx, ui_pop());
      render_frame(&ctx);
      ctx.frame_flag = 0;
    }
  }
}
```

Tiempo de juego = song_ms (timer0); el dibujo va en el main, nunca dentro de una ISR (el SPI del LCD es lento).

Buzzer pasivo
BUZZER_PWM · T14 (JD1)

PWM: f = nota
duty = volumen

EMIO (solo un cable en la PL)
TTC0_WAVE0_OUT → BUZZER_PWM

TTC0_WAVE0_OUT

TTC0 Triple Timer Counter
modo interval + match
(PWM)

regs

XTtcPs (xttcps.h)
SetInterval · SetMatch
SetPrescaler

f_Hz, vol %

audio.c · Buzzer_t
buzzer_tone(&bz, f_Hz, vol)
buzzer_tick(): avanza la melodía

play(&song), pause, stop, SFX

DebCfg* → debouncer_init()

buzzer_tick()

irq_gpio

xlconcat_0
4 → IRQ_F2P[3:0]

interrupt

axi_timer_0 · 1 ms
(auto-reload)

interrupt

axi_timer_1 · 2 ms
(auto-reload)

W: TLR0, TCSR0

W: TLR0, TCSR0

XTmrCtr (xtmrctr.h)
reset value 1 ms / 2 ms
auto-reload + interrupción

isr.c
timer0_isr (1 ms): song_ms++, buzzer_tick(), star_ms−−
timer1_isr (2 ms): joy_sample(); c/5: sensors_sample(); c/15: frame_flag (~33 FPS)
deb_isr · gpio_isr

song_ms, frame_flag

IRQ_F2P[3:0]

GIC (interrupciones)

ID 61–64

XScuGic (xscugic.h)
XScuGic_Connect(handlers)

_ISR()

microSD SDHC ≤ 32 GB · FAT32
BOOT.BIN · SONG1.SHC
RECORDS.TXT

CMD + DAT[3:0] · bloques de 512 B

SDIO0
controlador SD
MIO 40–47

sectores

xilffs (FatFs, ff.h) · XSdPs
f_mount · f_open · f_read
f_write · f_close

FRESULT, buffer

sd_storage.c · SdCtx
sd_load_song(&song, "0:/SONG1.SHC")
sd_load / sd_save_records(Record*)

Song*, Record[5]

SD0 y UART1 usan pines MIO: van directo al PS, sin pasar por la PL (no necesitan nada en Vivado).

PC · consola serial
USB-UART (J12)
115200 8N1

TX / RX (texto de configuración y logs)

UART1
MIO 48–49

bytes

xuartps · xil_printf
inbyte() · xil_printf()

chars

uart_config.c
pide por consola: DEB_MS, BTN_MASK
(→ DebCfg), JOY_UMBRAL, UMBRAL_SHAKE

JOY_UMBRAL, UMBRAL_SHAKE

AXI Interconnect (ps7_0_axi_periph) · AXI4-Lite

Leyenda

- IP / VHDL escrito por el grupo (Vivado)
- IP de librería Xilinx en la PL (Vivado)
- Bloque fijo del PS Zynq (se habilita en Vivado)
- Código C escrito por el grupo (Vitis)
- Driver Xilinx / código del curso (Vitis)
- Periférico o pin físico (Zybo / Booster)
- Externo: PC (consola UART) y microSD

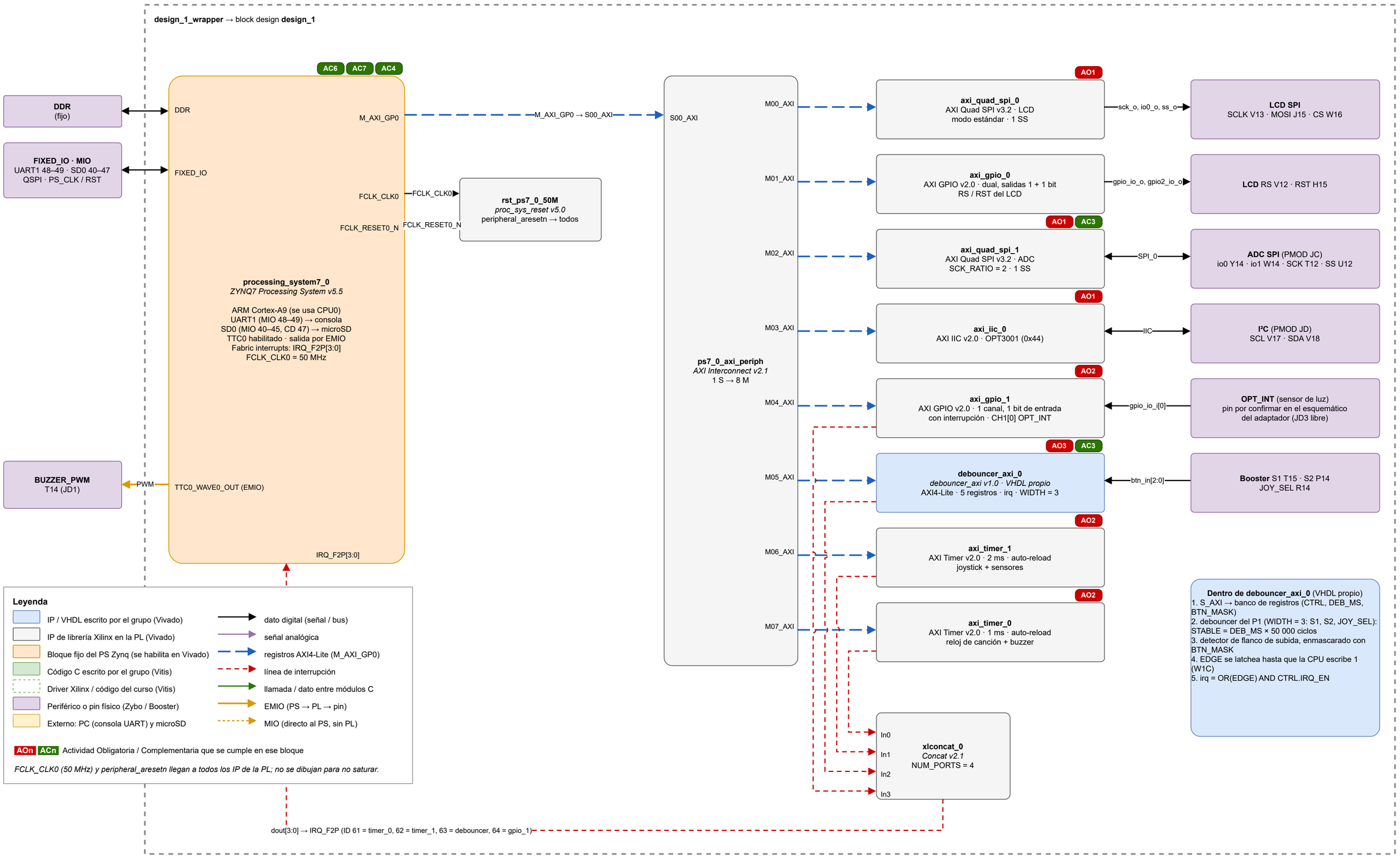
- dato digital (señal / bus)
- señal analógica
- registros AXI4-Lite (M_AXI_GP0)
- línea de interrupción
- llamada / dato entre módulos C
- EMIO (PS → PL → pin)
- MIO (directo al PS, sin PL)

AOn ACn Actividad Obligatoria / Complementaria que se cumple en ese bloque

FCLK_CLK0 (50 MHz) y peripheral_aresetn llegan a todos los IP de la PL; no se dibujan para no saturar.

Act.	Qué pide	Dónde se cumple en Sep Hero	Vivado / Vitis
AO1	Pantalla dinámica + 3 sensores de la booster + modificar 2 registros	render.c: pista ← ↑ ↓ → con receptores y notas-flecha, puntaje, barra de poder y rock meter (página 4) · acelerómetro (shake activa el poder) · sensor de luz (modo claro/oscuro) · micrófono (público y encore) · registros del OPT3001: CONFIG 0x01 y LOW_LIMIT 0x02 / HIGH_LIMIT 0x03	Ambos
AO2	≥ 2 interrupciones de timers hechos en Vivado + 1 interrupción del sensor de luz o temperatura	axi_timer_0 (1 ms: reloj de la canción y secuenciador del buzzer) · axi_timer_1 (2 ms: joystick; acelerómetro y mic cada 10 ms; frame_flag) · pin INT del OPT3001 → axi_gpio_1 → GIC (cambio de tema)	Vivado: IP + IRQ_F2P · Vitis: isr.c
AO3	IP Core propio que habla con un periférico y con el procesador; init con punteros; parámetros por UART	debouncer_axi_0 (VHDL, AXI4-Lite + irq) sobre S1, S2 y JOY_SEL de la booster · debouncer_init(Debouncer *dev, const DebCfg *cfg) con DEB_MS y BTN_MASK escritos en la consola (uart_config.c)	Ambos
AC1	Máquina de estados	game_fsm.c: INIT, MENU, RÉCORDS, CONFIG, SELECCIÓN, JUEGO, PAUSA, GAME OVER, más la sub-FSM del star power (página 3)	Vitis
AC2	Structs, arreglos, punteros, funciones y macros	Lane (enum), Note, Song, GameState, Buzzer_t, Debouncer, DebCfg, Record[5] · chart[], cola circular de hits y sprites const u16 arrow_spr[4][18*18] · macros HIT_WIN_MS, JOY_UMBRAL, STAR_STREAK, STAR_MS y offsets de registros	Vitis
AC3	2 periféricos adicionales (distintos de los de AO1)	Joystick (las 4 flechas del juego ← ↑ ↓ → y navegar los menús) y botones de la booster (S1 / S2 / JOY_SEL, filtrados por el IP propio)	Ambos
AC4	Boot de la Zybo	BOOT.BIN (FSBL + bitstream + sep_hero.elf) en la microSD, jumper JP5 en SD	Vitis (Create Boot Image)
AC5	VIO e ILA	*Pendiente: no se incluye en esta versión del diagrama*	—
AC6	Música con el buzzer	TTC0 → EMIO → BUZZER_PWM; audio.c toca la melodía de la canción sin bloquear (avanza en timer0_isr) y jingles de menú y game over	Ambos
AC7	Usar una memoria SD	SONG1.SHC (melodía + notas de la pista) y RECORDS.TXT (top 5) con xilffs	Vitis

Sep Hero — block design de Vivado (design_1) · ARM Cortex-A9 del PS, sin MicroBlaze



Mapa de direcciones (asignación típica de Vivado; confirmar en Address Editor)

IP	Base	Rango
axi_gpio_0	0x4120_0000	64 K
axi_gpio_1	0x4121_0000	64 K
axi_iic_0	0x4160_0000	64 K
axi_quad_spi_0	0x41E0_0000	64 K
axi_quad_spi_1	0x41E1_0000	64 K
axi_timer_0	0x4280_0000	64 K
axi_timer_1	0x4281_0000	64 K
debouncer_axi_0	0x43C0_0000	64 K

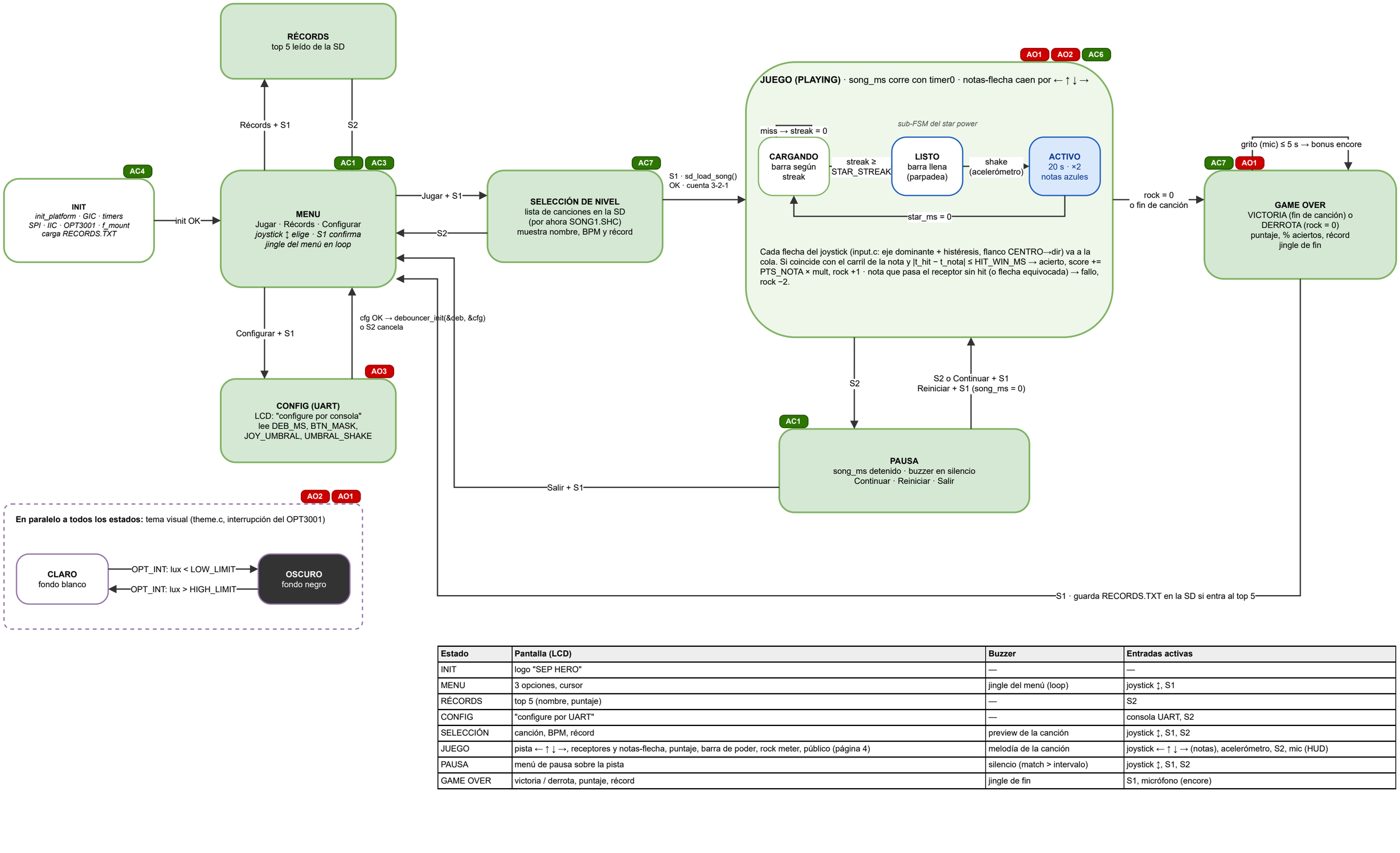
Registros de debouncer_axi_0 (AO3) · se configuran con debouncer_init(&deb, &cfg)

Offset	Nombre	Acceso	Contenido
0x00	CTRL	R/W	bit0 EN · bit1 IRQ_EN
0x04	DEB_MS	R/W	tiempo de antirrebote en ms (desde la UART)
0x08	BTN_MASK	R/W	[2:0] botones habilitados: S1, S2, JOY_SEL (desde la UART)
0x0C	STATE	R	[2:0] botones ya filtrados (nivel)
0x10	EDGE	R / W1C	[2:0] flancos de subida pendientes → irq

Interrupciones (GIC, sensibles a nivel alto)

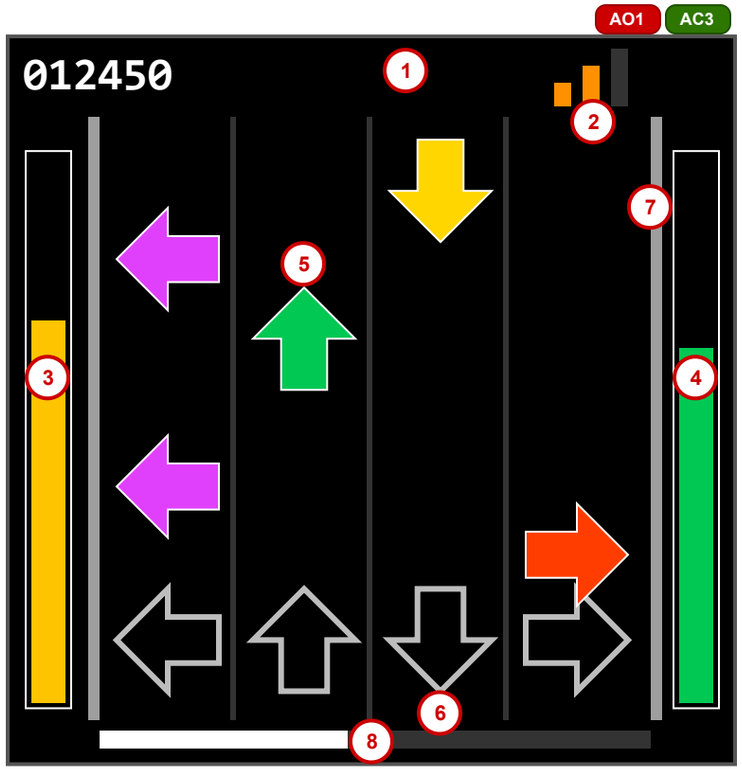
IRQ_F2P	ID	Fuente	Handler (isr.c)
[0]	61	axi_timer_0 (1 ms)	timer0_isr
[1]	62	axi_timer_1 (2 ms)	timer1_isr
[2]	63	debouncer_axi_0 (S1, S2, JOY_SEL)	deb_isr
[3]	64	axi_gpio_1 (OPT_INT)	gpio_isr

Sep Hero — máquina de estados del juego (game_fsm.c, Vitis)

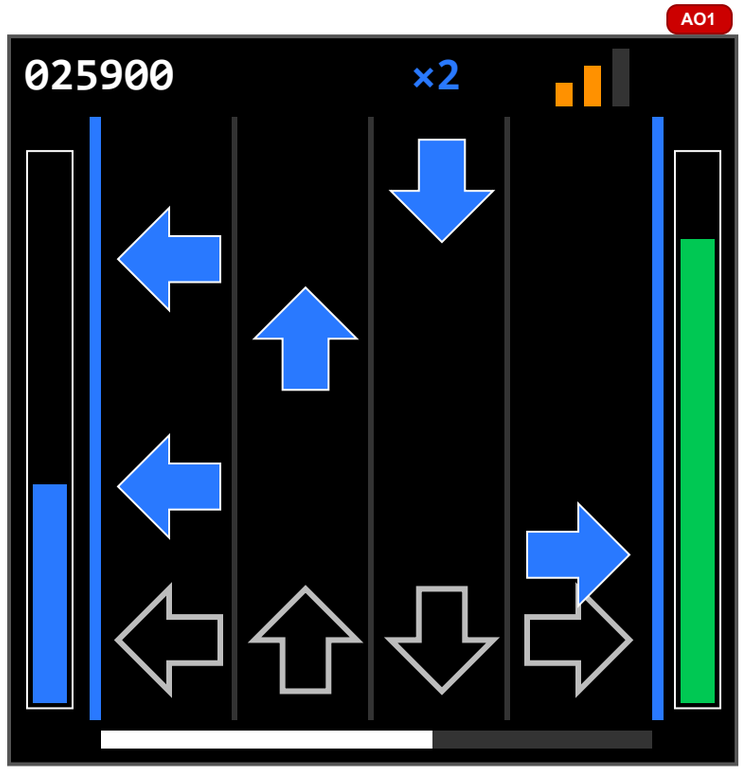


Estado	Pantalla (LCD)	Buzzer	Entradas activas
INIT	logo "SEP HERO"	—	—
MENU	3 opciones, cursor	jingle del menú (loop)	joystick ↕, S1
RÉCORDS	top 5 (nombre, puntaje)	—	S2
CONFIG	"configure por UART"	—	consola UART, S2
SELECCIÓN	canción, BPM, récord	preview de la canción	joystick ↕, S1, S2
JUEGO	pista ← ↑ ↓ →, receptores y notas-flecha, puntaje, barra de poder, rock meter, público (página 4)	melodía de la canción	joystick ← ↑ ↓ → (notas), acelerómetro, S2, mic (HUD)
PAUSA	menú de pausa sobre la pista	silencio (match > intervalo)	joystick ↕, S1, S2
GAME OVER	victoria / derrota, puntaje, récord	jingle de fin	S1, micrófono (encore)

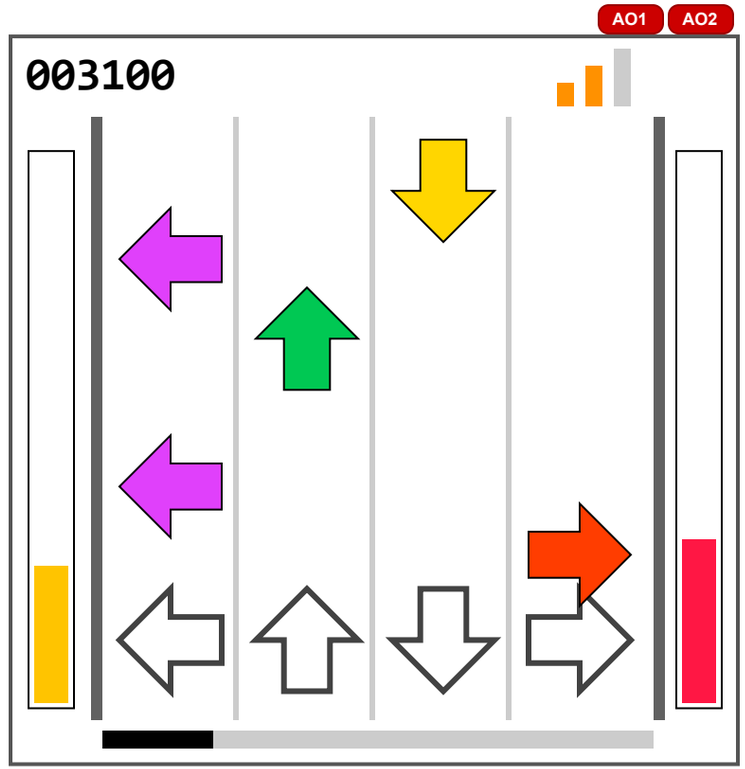
Sep Hero — pantalla de juego en el LCD 128×128 (dibujada a escala ×3)



Juego normal · tema OSCURO (poca luz)



STAR POWER activo · ×2 por 20 s · notas y bordes azules



Juego normal · tema CLARO (mucha luz) · rock meter bajo

Elementos de la pantalla (números en la primera pantalla)

1. Puntaje; aparece ×2 cuando el star power está activo.
2. Nivel del público (micrófono).
3. Barra de poder: se llena con aciertos seguidos (STAR_STREAK). Llena + agitar el mando (acelerómetro) → star power.
4. Rock meter: sube con aciertos y baja con fallos; en 0 → GAME OVER.
5. Notas-flecha que caen de arriba hacia abajo, cada una en su carril y con el color de su carril.
6. Receptores: contorno de ← ↑ ↓ → en la línea de golpe. Se acierta apuntando el joystick en esa dirección cuando la nota pasa sobre el receptor (±HIT_WIN_MS).
7. Bordes de la pista: grises normalmente, azules en star power.
8. Progreso de la canción (song_ms / duración).

El fondo negro / blanco lo decide el sensor de luz (interrupción del OPT3001).

Joystick → carril → color

Joystick	Carril (izq → der)	Lane	Color
← izquierda	1	IZQ = 0	magenta
↑ arriba	2	ARRIBA = 1	verde
↓ abajo	3	ABAJO = 2	amarillo
→ derecha	4	DER = 3	rojo
star power (todas las notas)			azul

Detección de la flecha (input.c, timer1 cada 2 ms): dx = x − 512, dy = y − 512 (ADC de 10 bit). Si max(|dx|, |dy|) > JOY_UMBRAL, la dirección es la del eje con mayor desviación (no hay diagonales). Cuenta como hit solo el paso CENTRO → dirección; para repetir hay que volver bajo JOY_UMBRAL − JOY_HIST. Verificar en la placa el signo del eje Y.
Dibujo: carriles de 24 px (x = 16–112), flechas de 18×18 px guardadas como sprites const u16 (RGB565) en flash. Solo se redibujan las zonas que cambian en cada frame_flag (~33 FPS), porque el SPI del LCD es lento.